PROJECT REPORT

Object Detection for Kitchen Tools using YOLOv11

Instructor: Dr. PHAM TRUNG KIEN
Group Members:

TRAN DUY KHANG - 104240557
TRAN KHANH DOAN - 104240544
DOAN THI MINH NGOC - 104240572
HUYNH NGOC PHUONG THAO - 104240794

December 13, 2025

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Proposed Solution . . . 3

2 Related Work **3**
- 2.1 Overview of Object Detection . . . 3
- 2.2 The YOLO Algorithm . . . 4
- 2.3 Why YOLOv11? . . . 4
- 2.4 Dataset Limitations and Proposed Solution . . . 4

3 Methodology **4**
- 3.1 System Overview . . . 4
- 3.2 Data Acquisition and Preprocessing . . . 5
  - 3.2.1 Data Sources . . . 5
  - 3.2.2 Annotation and Splitting . . . 5
- 3.3 Data Augmentation Strategy . . . 6
- 3.4 Model Configuration . . . 7

4 Experiments and Results **8**
- 4.1 Experimental Setup . . . 8
- 4.2 Training Results . . . 8
- 4.3 Model Evaluation . . . 9
  - 4.3.1 Precision-Recall Analysis . . . 9
  - 4.3.2 Confusion Matrix Analysis . . . 9
- 4.4 Qualitative Results . . . 10

5 Conclusion and Future Work **11**
- 5.1 Conclusion . . . 11
- 5.2 Limitations . . . 11
- 5.3 Future Work . . . 11

1 Introduction

1.1 Motivation

In the field of kitchen automation and cooking robots, development has been rapidly advancing. Modern intelligent systems not only assist humans in meal planning and ingredient tracking but are increasingly moving toward the ability to directly manipulate kitchen utensils. To perform such tasks effectively, the most crucial requirement is that the system must accurately recognize kitchen tools in real-time.

In addition to automation, safety is a paramount concern. There is a critical need for systems capable of detecting dangerous objects to provide special warnings, especially for households with children. By classifying utensils into safe and dangerous categories, the system can detect when a child comes into contact with hazardous tools (such as knives or scissors) and issue real-time alerts. This allows parents or supervisors to intervene promptly, preventing potential accidents.

Consequently, an AI recognition system serves not only automation purposes but also plays a vital safety role. Developing an AI model for kitchen-tool recognition therefore brings not only technological value but also significant social impact.

1.2 Problem Statement

The model needs to distinguish between many types of kitchen tools, even when they have similar shapes, are partially occluded, or are placed in complex environments mixed with various objects. Specifically, the system must address:

- **Diversity:** Many different shapes and materials.
- **Visual Similarity:** High similarities among materials.
- **Environment:** Varying lighting conditions.
- **Viewpoints:** Multiple angles and distances.
- **Background Clutter:** Numerous distracting objects.

1.3 Proposed Solution

This project proposes a "Kitchen Assistant" using the YOLOv11 Deep Learning model. The system acts as a pair of "smart eyes" to detect, classify and support the storage and retrieval process. The ultimate goal is to help users save time and ensure safety in the kitchen environment.

2 Related Work

2.1 Overview of Object Detection

In recent years, the field of computer vision has seen the rise of **Vision Transformers (ViTs)** and large-scale foundation models. While these modern architectures achieve state-of-the-art accuracy, they often come with a significant drawback: they require massive computational power and expensive hardware (high-end GPUs) to run efficiently.

For a practical kitchen assistant, relying on such heavy models is not feasible. The system needs to operate on standard personal computers or embedded devices with low latency. We cannot wait for a heavy model to process data on a cloud server when a safety hazard, such as a sharp knife, is detected. Therefore, the focus of this project remains on **efficient, real-time algorithms** that balance speed and accuracy, making the YOLO family the ideal choice over heavier modern alternatives.

2.2 The YOLO Algorithm

YOLO (You Only Look Once) is one of the most popular families of object detection models. Unlike older methods that look at the image multiple times, YOLO processes the entire image in a single pass. This "one-stage" approach allows the model to predict what the object classification is and where it is localized at the same time. This makes YOLO extremely fast and suitable for real-time video processing on personal computers.

2.3 Why YOLOv11?

YOLOv11 is the latest version released by Ultralytics. Compared to previous versions like YOLOv5 or YOLOv8, YOLOv11 has an optimized architecture. It is lighter, faster, and more accurate, while also requiring fewer computational resources. Most importantly, YOLOv11 performs better on small objects. In our project, items like spoons, scissors, and knives are very thin and small. Therefore, YOLOv11 is the best choice to solve this problem effectively.

2.4 Dataset Limitations and Proposed Solution

Public datasets such as the COCO dataset are very famous in the research community. However, they are too general for this specific project. For example, COCO contains a "knife" class, but it lacks specific Asian kitchen items like "Chopsticks" or "Rice Bowl". Furthermore, the images in public datasets are often clean and professional, which is different from a messy, real-world kitchen. To address this gap, this project builds a custom dataset with specific classes and realistic angles to ensure the system works well in practice.

3 Methodology

3.1 System Overview

The proposed system functions as a real-time object detection pipeline designed to identify specific kitchen tools. The workflow consists of three main stages: Data Acquisition, Model Inference, and Output Visualization. The overall architecture is illustrated in Figure **??**.

1. **Input Stage:** The system accepts input from a camera stream or static images. These raw images contain various kitchen items in complex backgrounds.
2. **Preprocessing & Inference Stage:**
    - First, input images are resized to 640×640 pixels to match the model's input requirement.
    - The **YOLOv11s** model then processes the image to extract features and predict bounding boxes along with class probabilities.
    - Non-Maximum Suppression (NMS) is applied to remove redundant overlapping boxes, keeping only the detection with the highest confidence score.

3. **Output Stage:** The final output renders bounding boxes around detected objects (e.g., Spoon, Knife, Pan) with their corresponding labels and confidence scores, providing immediate visual feedback to the user.

3.2 Data Acquisition and Preprocessing

3.2.1 Data Sources

To ensure diversity and model adaptability in real-world environments, the dataset was constructed from two primary sources:

- **Real-world data:** Images were collected directly from the deployment environment to ensure the model learns features that closely match reality.
- **Internet data:** Images were gathered from public online sources to introduce edge cases and diversify backgrounds and lighting conditions.

3.2.2 Annotation and Splitting

To ensure high-quality input data, we followed a strict annotation and splitting process:

- **Data Annotation:** We utilized the **Label Studio** platform to ensure label consistency and standardize the input format across the entire dataset.
- **Data Splitting:** After cleaning, the dataset was randomly split into two independent subsets with a **9:1 ratio** (90% for training and 10% for validation). This ratio was selected to maximize the amount of data available for learning complex features, while retaining a sufficient amount of data to objectively evaluate performance and prevent overfitting during training.

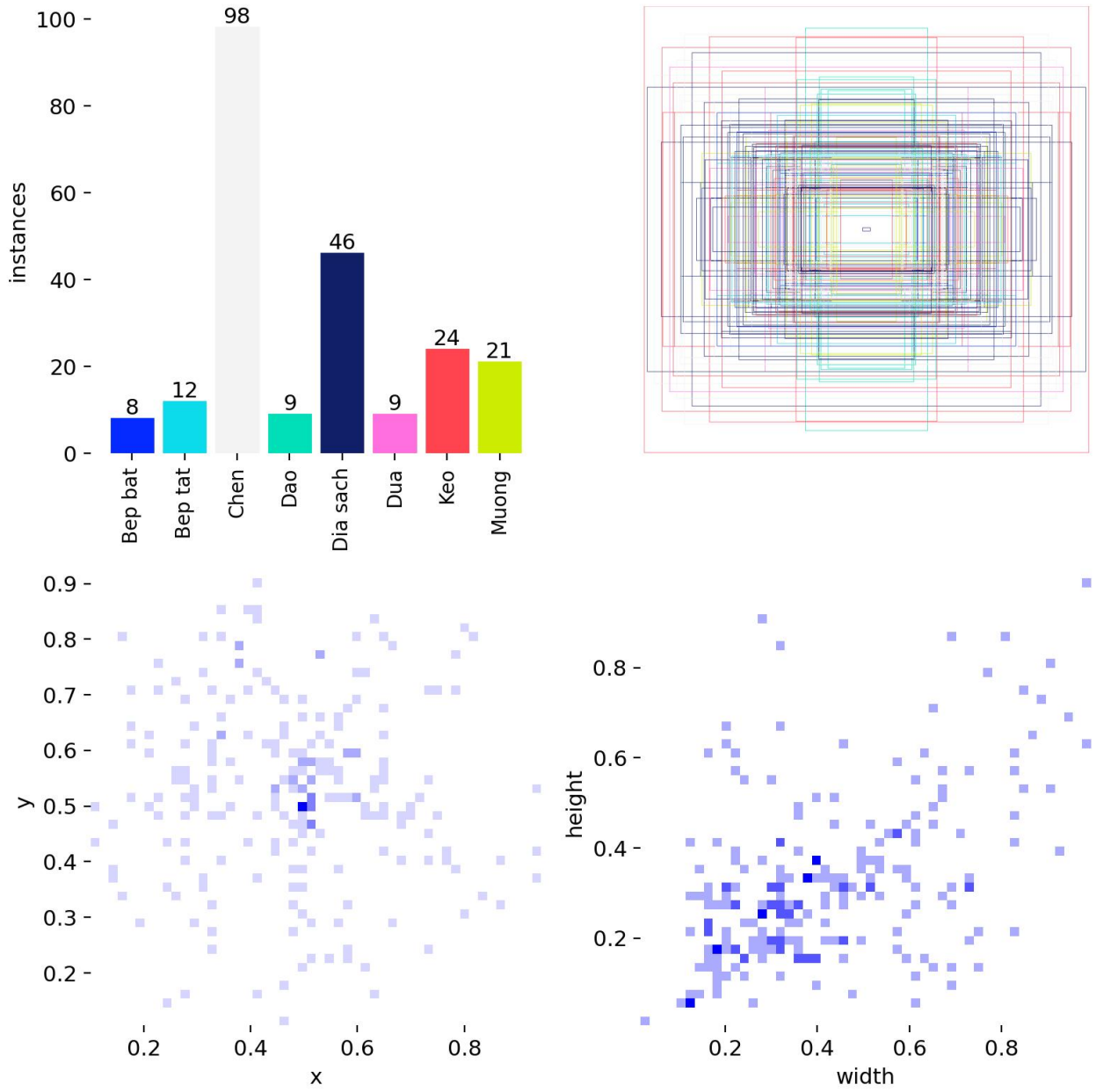


Figure 1: Dataset statistics visualization: Instance counts per class and bounding box dimensions.

3.3 Data Augmentation Strategy

To enhance the model's generalization capability and prevent overfitting, especially given the limited size of the custom dataset, we applied online data augmentation techniques during training. The specific hyperparameters are detailed in Table 1.

Table 1: Hyperparameters configuration for Data Augmentation.

Parameter	Value	Description
Mosaic	1.0	Stitches 4 training images into 1 (100% probability).
Scale	0.5	Randomly scales images by ±50%.
Translate	0.1	Shifts images horizontally/vertically by ±10%.
HSV-H	0.015	Adjusts Hue (Color tone).
HSV-S	0.7	Adjusts Saturation (Color intensity).
HSV-V	0.4	Adjusts Value (Brightness).
FlipLR	0.5	Flips images horizontally with 50% probability.
Close Mosaic	10	Disables Mosaic augmentation in the last 10 epochs.
Degrees	0.0	No rotation applied to maintain realistic orientation.

Key techniques include **Mosaic**, which helps the model detect small objects (like spoons or knife tips) by forcing it to learn from context, and **HSV augmentation**, which ensures the model is robust against different lighting conditions in various kitchens.

3.4 Model Configuration

Based on the experimental requirements and hardware constraints, the model was trained using the **Ultralytics YOLOv11** framework. The specific hyperparameters extracted from the training configuration are detailed in Table 2.

Table 2: Hyperparameters configuration for YOLOv11s training.

Parameter	Value
Model Architecture	YOLOv11s (Small version)
Pre-trained Weights	True (yolo11s.pt)
Input Image Size	640×640 pixels
Epochs	60
Batch Size	16
Optimizer	Auto (Defaults to SGD)
Initial Learning Rate ($lr0$)	0.01
Momentum	0.937
Weight Decay	0.0005
Patience	100 epochs (Early Stopping)

Configuration Analysis:

- **Model Selection:** We selected **YOLOv11s** because it offers an optimal balance between detection speed and accuracy, making it suitable for real-time kitchen tool detection.
- **Training Strategy:** The model was trained for **60 epochs** with a batch size of **16**. We utilized transfer learning by loading pre-trained weights ('pretrained: true'), which significantly reduces training time and improves performance on our custom dataset.
- **Optimization:** An initial learning rate of 0.01 with a momentum of 0.937 was used to ensure stable convergence. The patience parameter was set to 100 to prevent overfitting, although the training completed the full 60 epochs successfully.

4 Experiments and Results

4.1 Experimental Setup

To ensure the reproducibility of the results and handle the computational load of deep learning, the project was implemented using the following environments:

- **Google Colab:** This cloud-based platform was utilized as the primary environment for the **training phase**. Colab provided free access to NVIDIA GPUs (specifically the Tesla T4), which significantly accelerated the training process of the YOLOv11 architecture compared to local CPU execution.
- **Anaconda Distribution:** We used Anaconda to manage the local development environment. It was primarily used for **data preprocessing, annotation**, and running real-time inference tests. Anaconda helped in creating an isolated virtual environment to manage dependencies such as Python 3.9, PyTorch, OpenCV, and the Ultralytics library without conflicts.

4.2 Training Results

The model was trained for 60 epochs on the custom dataset. The training history, including loss values and performance metrics, is visualized in Figure 2.

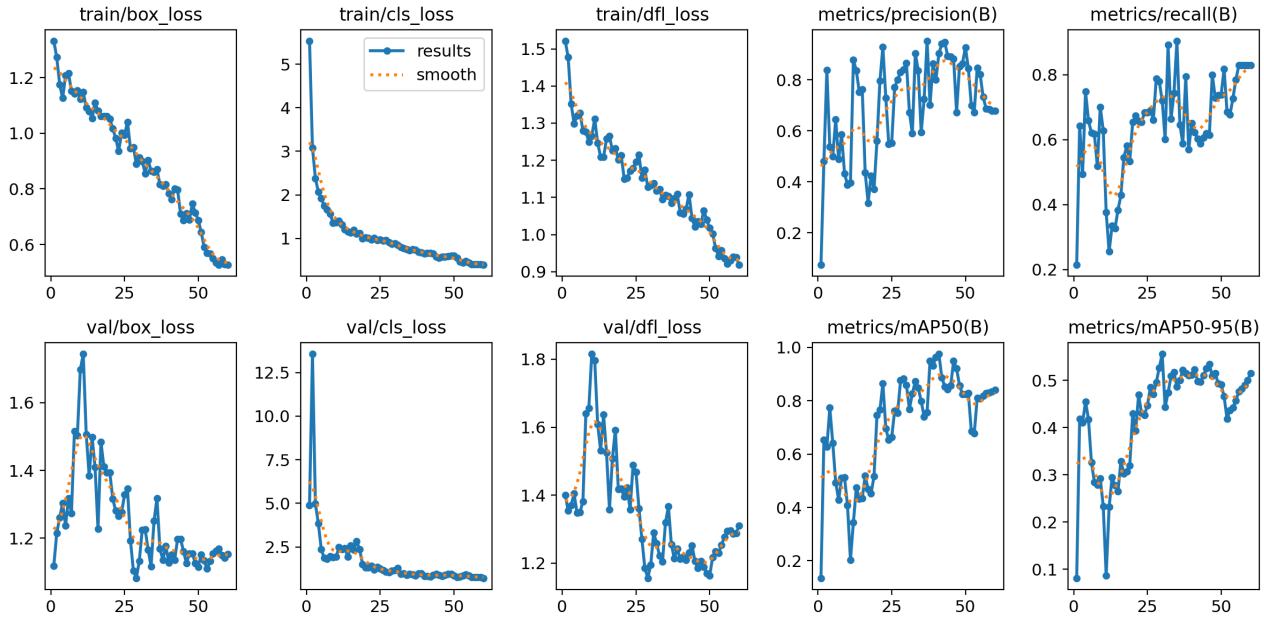


Figure 2: Training curves showing decreasing loss and increasing accuracy over 60 epochs.

As observed, both Box Loss and Classification Loss decreased steadily, indicating that the YOLOv11s model successfully learned to localize and classify the kitchen tools. The mean Average Precision (mAP@0.5) increased significantly in the first 20 epochs and stabilized around 0.85, demonstrating good convergence without signs of severe overfitting.

4.3 Model Evaluation

4.3.1 Precision-Recall Analysis

To evaluate the overall detection performance, we analyzed the Precision-Recall curve. As shown in Figure 3, the model achieved an **mAP@0.5 of 0.858** across all classes.

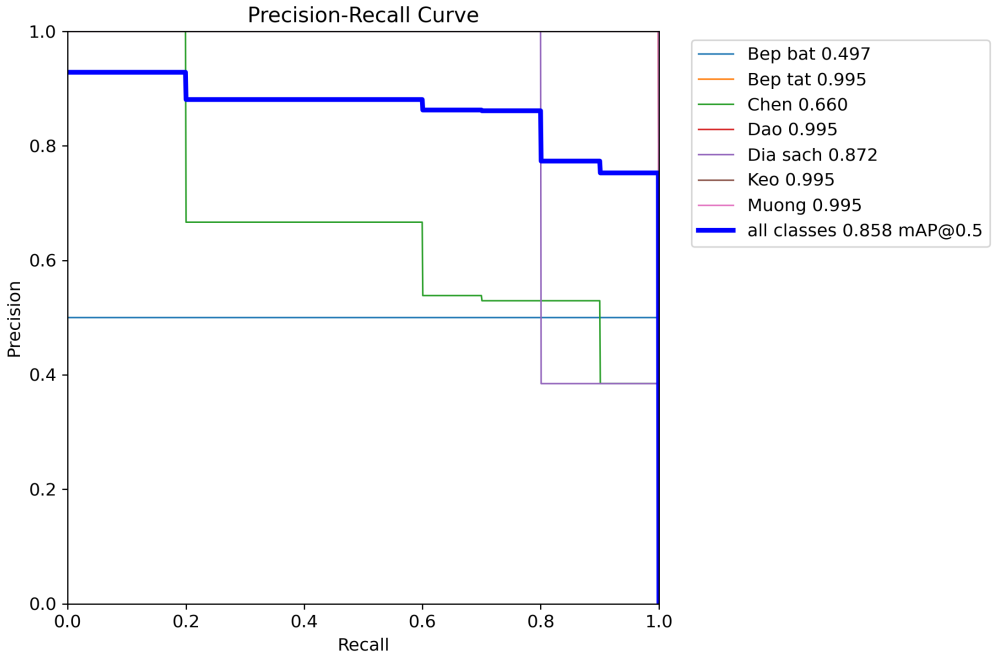


Figure 3: Precision-Recall Curve achieving 0.858 mAP@0.5.

Classes such as "Dao" (Knife), "Muong" (Spoon), and "Bep tat" reached near-perfect precision (0.995). However, the "Bep bat" class showed lower performance (0.497), likely due to the complexity of the object's shape or insufficient training samples.

4.3.2 Confusion Matrix Analysis

The Normalized Confusion Matrix (Figure 4) provides a detailed insight into classification errors.

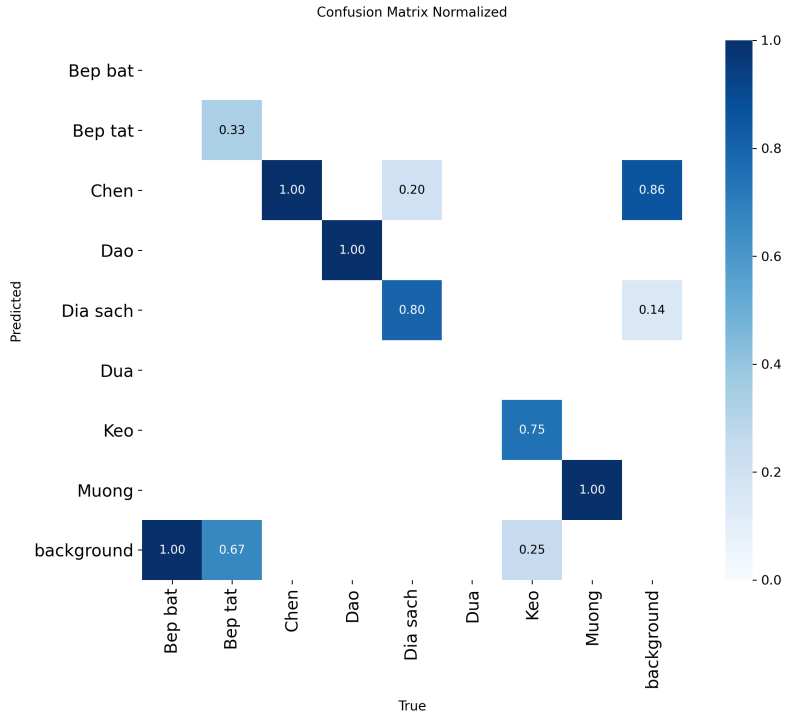


Figure 4: Normalized Confusion Matrix showing prediction accuracy per class.

Key observations:

- **High Accuracy:** The model performs perfectly on rigid and distinct objects like "Chen" (1.00) and "Dao" (1.00).
- **Misclassification:** The "Keo" (Scissors) class has an accuracy of 0.75, with 25% of instances being confused with the background. This suggests that the model sometimes fails to detect scissors when they are placed in cluttered environments or have low contrast with the background.
- **Background Confusion:** "Dia sach" also shows an 14% confusion rate with the background.

4.4 Qualitative Results

To visually demonstrate the model's capability, Figure 5 displays the detection results on the test set. The model accurately draws bounding boxes around kitchen tools even in challenging lighting conditions.

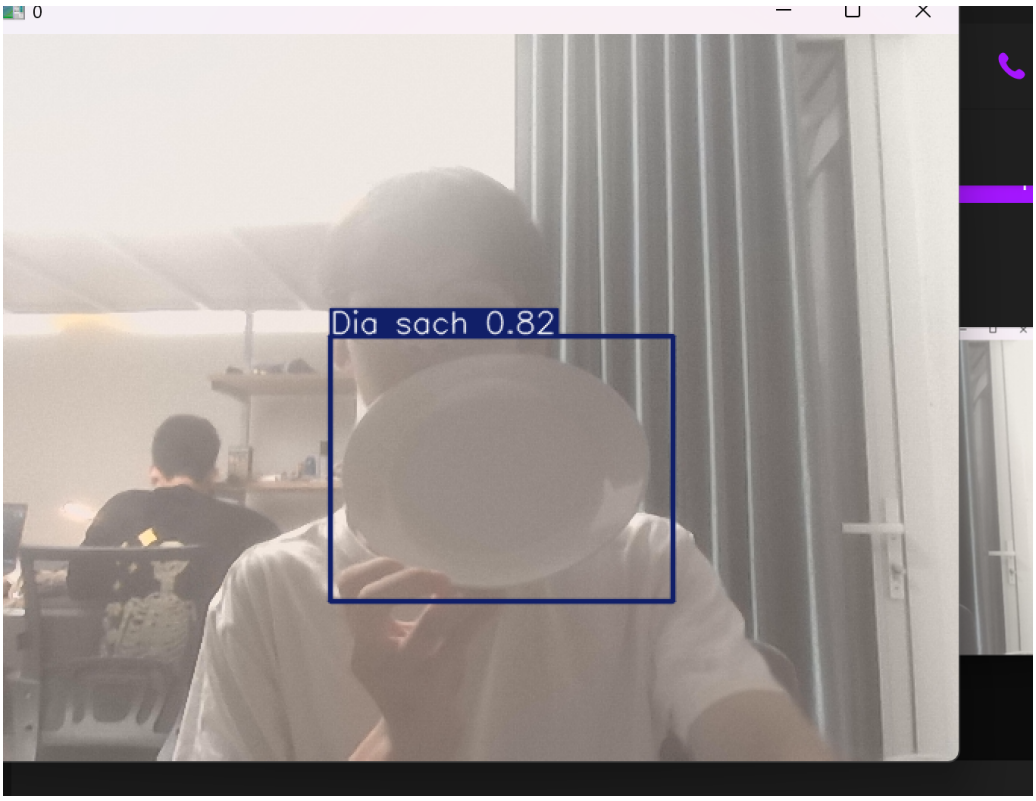


Figure 5: Qualitative results: Model detection on unseen images.

5 Conclusion and Future Work

5.1 Conclusion

In this project, we successfully developed a real-time kitchen tool detection system using **YOLOv11s**. By constructing a diverse dataset combining real-world and internet images, the model achieved a promising performance with an **mAP@0.5 of 85.8%**. The system effectively identifies common items like knives, spoons, and bowls, proving the feasibility of AI-assisted kitchen organization.

5.2 Limitations

Despite the positive results, the system has some limitations:

- **Class Imbalance:** As seen in the confusion matrix, classes with fewer samples (e.g., "Keo", "Bep bat") have lower accuracy compared to dominant classes like "Chen".
- **Background Clutter:** Thin objects like scissors are occasionally missed (False Negatives) when placed on complex backgrounds.

5.3 Future Work

To improve the system, we propose the following directions:

- **Data Augmentation:** Collect more images for under-represented classes ("Keo", "Bep bat") and apply advanced augmentation techniques (MixUp, Mosaic) to improve robustness.

- **Deployment:** Integrate the model into a mobile application or a web-based platform to provide a user-friendly interface for real-time inventory management.

References

[1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," Version 8.0.0, 2023. [Online]. Available: `https://github.com/ultralytics/ultralytics`

[2] Heartex, "Label Studio: Open Source Data Labeling," 2024. [Online]. Available: `https://labelstud.io/`

[3] Van Rossum, G. and Drake, F.L., "Python 3 Reference Manual," Scotts Valley, CA: CreateSpace, 2009.